

Labwork 6 - Map

Ho Thanh Thuy Tien

October 2, 2026

1 Implementation

For this labwork, I will implement several basic image-processing operations on the GPU using CUDA through Numba which are brightness control, grayscale image binarization and blending two images.

Grayscale Image Binarization The original input image is an RGB image. The first step is to convert it to grayscale. For every pixel, the CUDA kernel reads the red, green, and blue channels and computes their average:

$$G(x, y) = \frac{R(x, y) + G(x, y) + B(x, y)}{3}. \quad (1)$$

After obtaining the grayscale image, I apply a threshold with value equal to 128. Here, each grayscale pixel is compared with this threshold. If the pixel value is smaller than 128, the output becomes 0. Otherwise, it becomes 1. Therefore, we have a result like this:

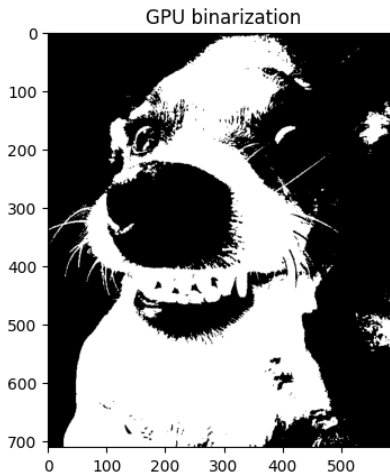


Figure 1: GPU binarization

Brightness Control Brightness control is implemented by adding a constant value to every color channel of every pixel. In this case, I choose brightness with value equal to 60. For each color channel, the original pixel value is added to the selected brightness adjustment (value). Since an 8-bit image must have pixel values within the range $[0, 255]$, the resulting value is clipped to this range before being stored in the output image:

```
for c in range(src.shape[2]):
    v = int(src[tidy, tidy, c]) + value
    if v < 0:
        v = 0
    elif v > 255:
        v = 255
    dst[tidy, tidy, c] = v
```

I tested both positive and negative brightness values and here is the result:

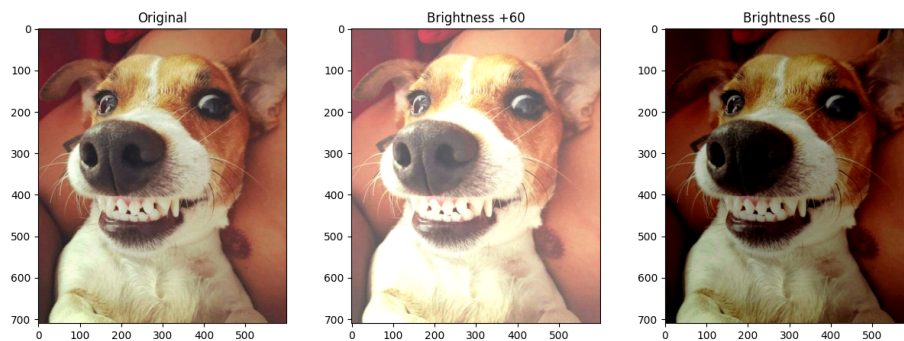


Figure 2: GPU brightness control

Blending Two Images For image blending, a second image is loaded with PIL and resized to the same width and height as the first image. The second image is shown below:



Figure 3: Second image

After that, both images are copied to GPU memory. The blending operation follows the formula presented in the slides, where the coefficient c determines the weight of each input image:

$$Q(x, y) = c \times \Phi_1(x, y) + (1 - c) \times \Phi_2(x, y) \quad (2)$$

With a coefficient of 0.5, both images contribute equally to the result. I also tested coefficients 0.25, 0.50, and 0.75 as shown in Figure 4

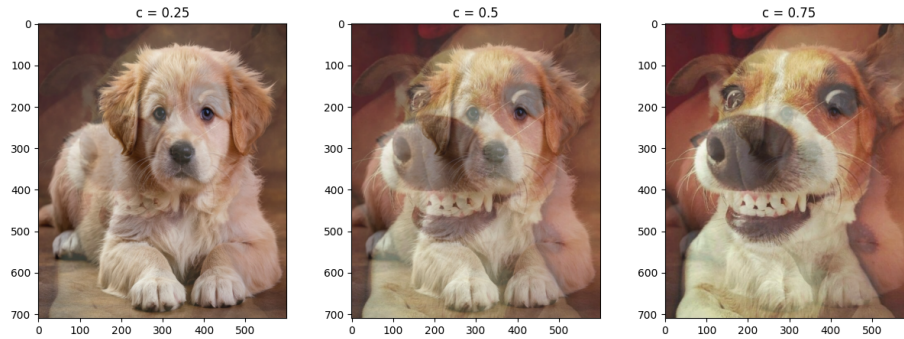


Figure 4: GPU blend two images

I also converted both input images to grayscale and blended the grayscale results using the same blending kernel. The result is shown in the Figure 5

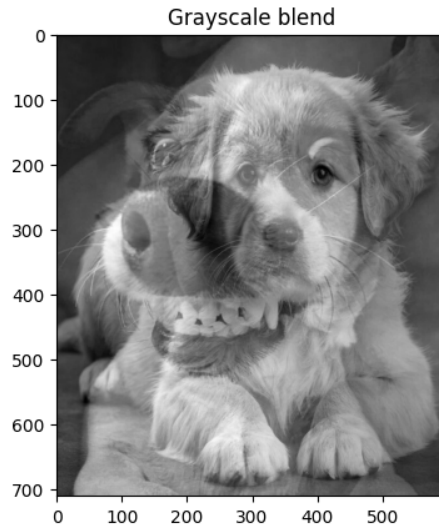


Figure 5: GPU grayscale blend

2 Experiment Result

To study the effect of CUDA block configuration, I tested the following two-dimensional block sizes: (8,4), (16,4), (8,16), (16,16), (32,16), (32,32).

The experiment demonstrates that CUDA execution time varies depending on the 2D block configuration, as smaller blocks can create more blocks and increase scheduling overhead. In this experiment, brightness achieved its lowest time of approximately 0.000128 seconds with a 32×32 block, while blending performed best at approximately 0.000278 seconds with a 32×16 block. Grayscale binarization achieved its lowest time of approximately 0.000210 seconds using a 16×16 block. Therefore, no single block size was optimal for all kernels. Performance depends on the computation, memory access patterns, scheduling, and measurement variation, so testing multiple configurations is important when optimizing CUDA programs.

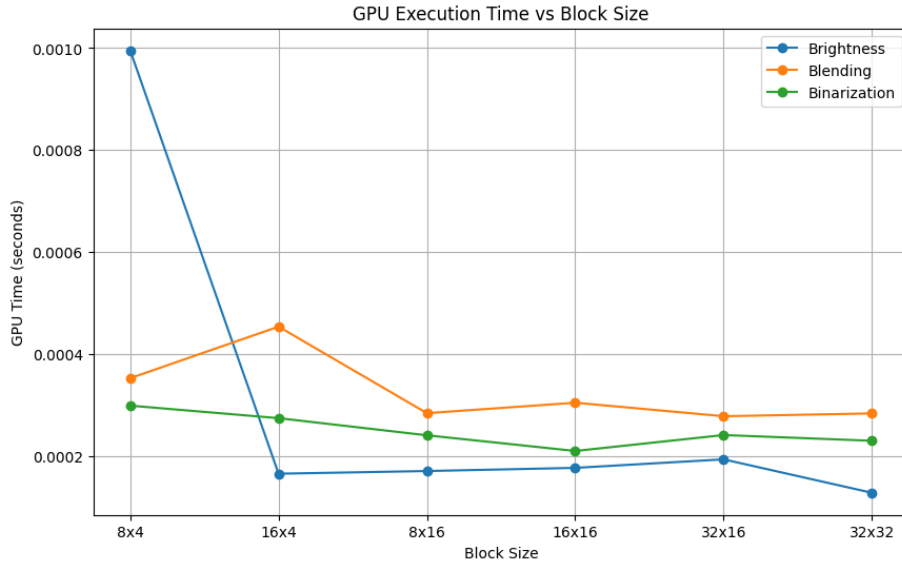


Figure 6: GPU Execution Time vs Block Size for Brightness, Blending, and Grayscale Binarization